Q3. How do the other experimental factors, i.e., the victim DRL model's reward criterion, the hyper-parameter coupling, the length of state-action sequence, and the number of shadow DRL models influence the attack?
Q4. How to mitigate the risk of hyper-parameter leakage?

A. Experimental Setup

Environments: We choose the "CartPole", "Acrobot", "Ant", and "HalfCheetah" environments in the Gym.[3] We believe that the environments are representative: 1) Gym is a mainstream platform that provides a unified interface for training and testing the DRL models. 2) Selected environments are commonly used in academia to evaluate the effectiveness of attacks against DRL models [8], [30], [52].

Targeted DRL model: We adopted DQN [47], PG [76], PPO [23], [59], TRPO [22], and SAC [58] models during evaluation. The DQN, PG, and PPO models represent three basic ideas of the RL models, i.e., value-based strategy, policy-based strategy, and actor-critic strategy. Most of the existing model-free RL methods are evolved from these three approaches. We select TRPO and SAC to represent the advanced RL models.

Targeted hyper-parameters: Table I shows the values of hyper-parameters. The first three hyper-parameters are about the architecture of the DRL policy network, and the others are related to the optimization process. For each hyper-parameter, we choose three different values, which are usually the default selection in open-sourced frameworks [12], [24], e.g., ReLU and the Adam optimizer.

Shadow model contruction: For each combination, we collect 20 shadow DRL models for each hyper-parameter setting (totally 20×3^6), and then divide the shadow models into a training set (16×3^6) and a test set (4×3^6), which is the default setup for the subsequent experimental steps. Since it is difficult to train a DRL model satisfying the reward criterion under some hyper-parameter values [46], the distributions of hyper-parameter values are possibly not uniform among the shadow DRL models. The distributions of the hyper-parameters are shown in Appendix B, available online.

Evaluation metric: In our experiment, we follow the metric commonly used to evaluate inference attacks, measuring the accuracy of prediction [8], [49], [51]. We utilize shadow models in the training set to generate state-action sequences and train the attack model. Then, we take the shadow models in the test set as the victim model and infer the hyper-parameter settings based on their state-action sequences.

Methods: We compare the inference performance between the passive state generation and the active state generation methods. "passive_env" (Algorithm 2) uses the states directly from the interaction between the DRL model and the environment. "active_env" optimizes the states from "passive_env" by leveraging the state optimization in Algorithm 3. For both methods, the default sequence size is 200 in the experiments. The baseline method, i.e., "majority_vote", adopts the largest proportions as the prediction for the victim DRL model.

Experimental settings: We adopt the ResNet framework as our attack model and illustrate its architecture in Appendix A, available online. We train the attack model for 400 epochs. For the passive method, we use the SGD optimizer to train the attack model with a learning rate of 0.01. We leverage an additional Adam optimizer with a learning rate of 0.01 to tune the states every 10 epochs during the attack model training.

B. Overall Performance

We show the effectiveness of HyperInfer in six combinations of basic DRL models and Gym environments.

Setup: We use the reward criteria in OpenAI Gym leaderboard[4] for the victim DRL models, which are 500 for Cartpole and -100 for Acrobot, respectively. In Fig. 3, we vary the types of hyper-parameter on the X-axis and show the corresponding inference accuracy on the Y-axis. Each histogram in the plots shows the average inference accuracy of 3 repeated experiments with the standard deviation.

Observations: We have the following observations from Fig. 3. 1) The inference accuracy of HyperInfer is all higher than the baseline method, which means that the behaviors of DRL models can indeed expose the hyper-parameter settings. For example, the inference accuracy for the activation function is beyond 60% across the experiment settings, even exceeding 90% on PPO.

2) HyperInfer obtains different inference accuracy over various DRL models. The adversary's inference effectiveness on the PG and PPO models is better than that of the DQN models for the hyper-parameters optimizer and learning rate. For the PG and the PPO models, HyperInfer achieves a huge boost in accuracy on the optimizer and the learning rate, while HyperInfer slightly surpasses the baseline method for the DQN model. Recalling Section II-B, the DQN models use the policy network to approximate the Q-table in the training process and choose the action according to the formed Q-table, i.e., indirectly act. PG and PPO use the policy network to directly build the relation between states and actions without the Q-value approximation in DQN, i.e., directly act. Thus, it is more difficult to infer the optimization-related hyper-parameters from DQN.

3) The inference accuracy of HyperInfer varies over the hyper-parameters. In Fig. 3, HyperInfer usually obtains higher inference accuracy on the hyper-parameter activation function compared to the hyper-parameter learning rate. As a structural hyper-parameter, the different activation functions have unique features. For example, the Tanh activation function is symmetric about (0,0), while the ReLU and ELU activation functions are asymmetrical. In addition, the functional features of activation do not change once the DRL model is built. Therefore, the activation function of the DRL model is at increased risk of leakage. The hyper-parameter learning rate controls the update speed of the models' weights. Compared to the activation function, the learning rate guides the model's actions by affecting the model's weight values. Thus, the features of the learning rate become blurred as the training process.

[3] https://www.gymlibrary.ml/

[4] https://github.com/openai/gym/wiki/Leaderboard